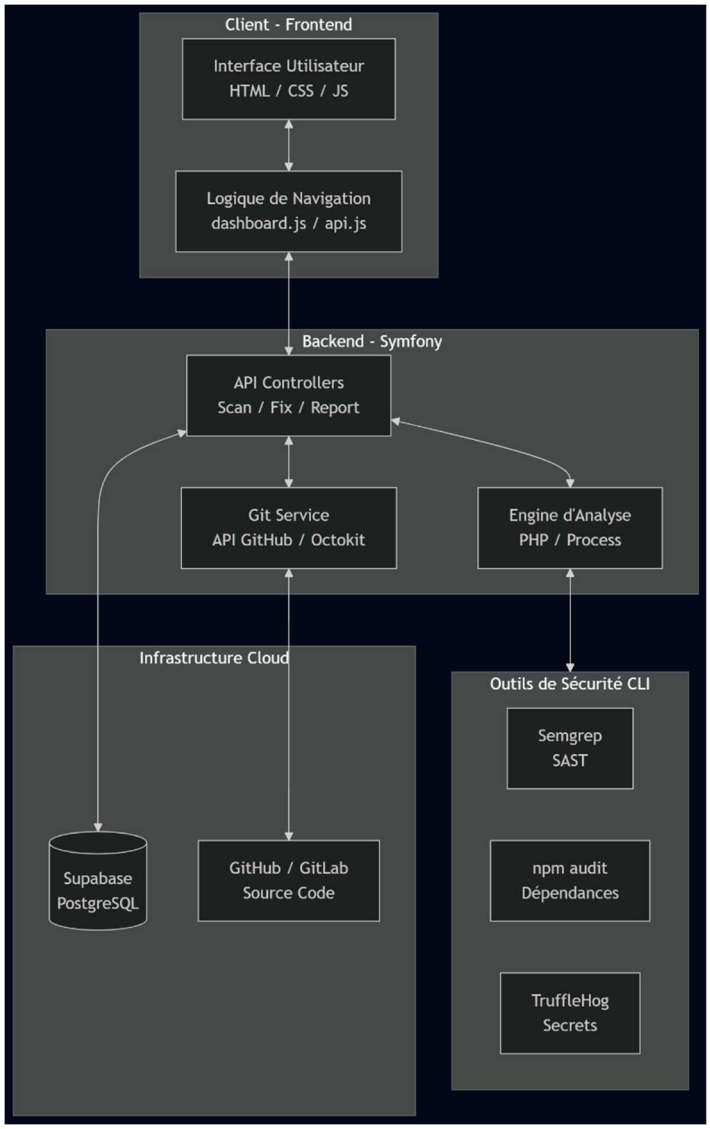
Dossier Technique

1. Schéma d'Architecture Système

Notre plateforme suit une architecture découplée pour garantir la séparation des responsabilités entre l'interface utilisateur, la logique d'analyse et le stockage.



2. Choix Techniques Justifiés

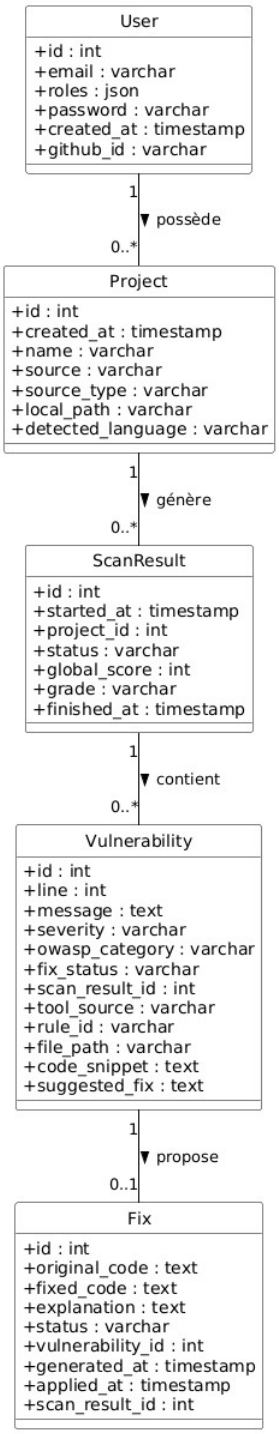
Pour répondre aux contraintes du Hackathon et aux besoins de CyberSafe Solutions, nous avons opté pour la stack suivante :

- **Frontend (HTML/CSS/JS) :** Choix de la légèreté. Pas de framework lourd pour garantir une interface ultra-rapide et se concentrer sur l'intégration de l'API REST.
- **Backend (Symfony / PHP 8) :** Framework robuste et structuré, idéal pour développer une API sécurisée, orchestrer des processus systèmes (CLI) et gérer les requêtes HTTP complexes (clonage Git).
- **Base de données (Supabase / PostgreSQL) :** Solution cloud performante permettant un déploiement instantané et une gestion relationnelle forte pour lier les vulnérabilités à leurs projets.
- **Moteur d'analyse (Semgrep, npm audit, TruffleHog) :** Nous avons choisi ces trois outils open-source de référence car ils sont complémentaires (SAST pour la logique, CVE pour les dépendances, analyse d'entropie pour les secrets) et s'exécutent très bien en ligne de commande.

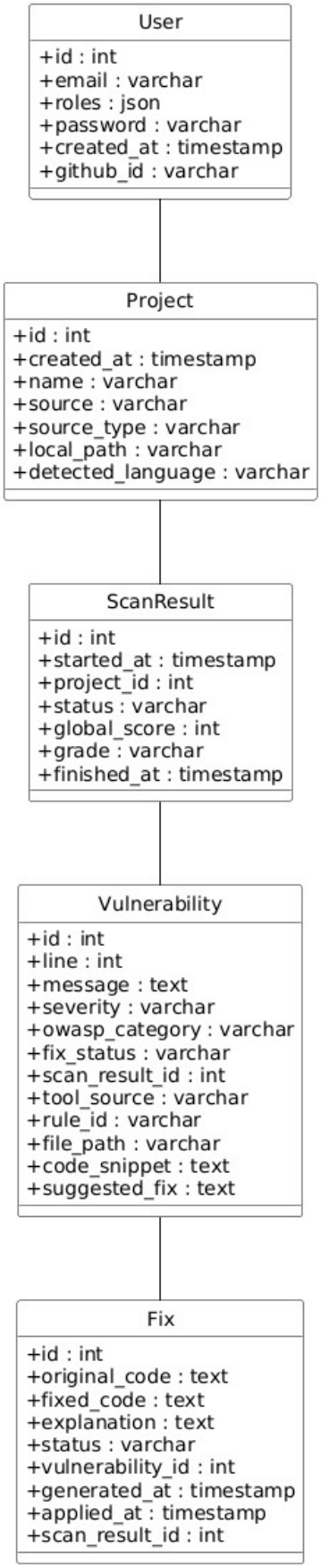
3. Schéma de la Base de Données (MLD/MCD)

Notre base de données relationnelle est construite pour tracer précisément l'historique des analyses et l'état des correctifs.

MCD :



MLD :


User
+id : int
+email : varchar
+roles : json
+password : varchar
+created_at : timestamp
+github_id : varchar
Project
+id : int
+created_at : timestamp
+name : varchar
+source : varchar
+source_type : varchar
+local_path : varchar
+detected_language : varchar
ScanResult
+id : int
+started_at : timestamp
+project_id : int
+status : varchar
+global_score : int
+grade : varchar
+finished_at : timestamp
Vulnerability
+id : int
+line : int
+message : text
+severity : varchar
+owasp_category : varchar
+fix_status : varchar
+scan_result_id : int
+tool_source : varchar
+rule_id : varchar
+file_path : varchar
+code_snippet : text
+suggested_fix : text
Fix
+id : int
+original_code : text
+fixed_code : text
+explanation : text
+status : varchar
+vulnerability_id : int
+generated_at : timestamp
+applied_at : timestamp
+scan_result_id : int

4. Stratégie de Mapping : OWASP Top 10 (2025)

C'est le cœur de notre logique métier. Le backend Symfony parse les retours JSON des trois analyseurs et les associe dynamiquement aux catégories de l'OWASP Top 10 : 2025 via le tableau de correspondance suivant :

Outil Source	Faille Détectée (Exemples)	Catégorie OWASP 2025 Correspondante
Semgrep	Requêtes SQL non préparées, failles XSS	**A03: Injection**
Semgrep	Mauvaise gestion des sessions, mots de passe en clair	**A07: Identification and Authentication Failures**
Semgrep	Chiffrement faible, données sensibles exposées	**A02: Cryptographic Failures**
npm audit	Bibliothèques obsolètes ou contenant des CVE connues	**A06: Vulnerable and Outdated Components**
TruffleHog	Clés API GitHub, tokens AWS, mots de passe hardcodés	**A01: Broken Access Control** (ou A02 selon le contexte du secret)